

Lista de Exercícios I

Prof.: Arthur Fernandes - Departamento de Informática

1 Instruções

Esta lista de exercícios possui 14 questões valendo, ao total 15 pontos **extras**. As respostas devem ser **manuscritas** em folha com pauta e entregues ao professor no dia da prova.

1. (1 point) Escreva a função `def identifica_triangulo(a, b, c)` que recebe como parâmetros o tamanho dos três lados a, b e c de um triângulo e retorna uma tupla contendo:
 - `True` ou `False` indicando se os valores informados formam triângulo válido.
 - O tipo do triângulo segundo o tamanho dos seus lados (isósceles, escaleno e equilátero).
2. (1 point) Escreva uma função que recebe via `input()`, dois valores *lat* e *long*, realiza a validação dos dados com loop de validação seguindo as seguintes regras:
 - Latitude deve ser um `float` no intervalo $[-90, 90]$.
 - Longitude deve ser um `float` no intervalo $[-180, 180]$.
 - O programa não deve aceitar valores inválidos (ex.: letras, símbolos, etc).

A função deve retornar a tupla (*lat*, *long*).

3. (1 point) Escreva um programa que receba via `input()` um inteiro n e calcula a soma dos n primeiros números primos e imprima na tela. O programa deve ter duas funções definidas:
 - `def soma_primos(n)`: deve retornar a soma dos n primeiros primos.
 - `def checa_primo(x)`: deve retornar `True` ou `False` indicando se o parâmetro x é primo.
4. (1 point) Escreva um programa que recebe um inteiro n via `input()` e gera uma lista de inteiros com n números inteiros aleatórios. O programa deve definir a função `def gera_lista(n)` para modularizar a geração da lista com n inteiros.

Obs.: Para gerar um inteiro aleatório no intervalo $[x, y]$ utilize `random.randint(x, y)`.

5. (1 point) Escreva um programa que *printa* na tela o maior e o menor número de uma lista de 300 inteiros.

Obs.: Utilize apenas laços de repetição, não será permitido uso da função `max()` ou similares. Para obter a lista, utilize a função `def gera_lista(n)`, que retorna uma lista de n inteiros aleatórios.

6. (1 point) Escreva um programa que *printa* na tela quantos números de uma lista estão dentro do intervalo $[25, 70]$.

Obs.: Para obter a lista, utilize a função `def gera_lista(n)`, que retorna uma lista de n inteiros aleatórios.

7. (1 ½ points) Escreva um programa que faça o *deslocamento* para a direita, de uma lista de 10 elementos. O deslocamento troca os índices dos elementos de maneira que o elemento 0 se torna o elemento 1, o elemento 1 se torna o elemento 2, e assim sucessivamente. O último elemento da lista vira o elemento 0. A seguir um exemplo de deslocamento de lista para a direita.

0	1	2	3		0	1	2	3
12	15	7	8	→	8	12	15	7

Obs.: Para obter a lista, utilize a função `def gera_lista(n)`, que retorna uma lista de n inteiros aleatórios.

8. (1 point) Escreva um programa que recebe dois inteiros m e n via `input()` e gera uma matriz de inteiros aleatórios de tamanho $m \times n$. O programa deve definir a função `def gera_matriz(m, n)` para modularizar a geração da matriz $m \times n$.

Obs.: Para gerar um inteiro aleatório no intervalo $[x, y]$ utilize `random.randint(x, y)`.

9. (1 ½ points) Escreva um programa que atualiza os valores em memória de uma matriz $M_{m \times n}$ de modo que a matriz resultante seja a sua transposta. Não será permitido criar outra matriz para auxiliar o cálculo.

Obs.: Para obter a matriz, utilize a função `def gera_matriz(m, n)` que gera uma matriz $m \times n$ de inteiros aleatórios.

10. (1 point) Escreva um programa que calcula a soma dos números da diagonal **secundária** de uma matriz de inteiros $M_{5 \times 5}$.

Obs.: Para obter a matriz, utilize a função `def gera_matriz(m, n)` que gera uma matriz $m \times n$ de inteiros aleatórios.

11. (1 point) Escreva a função `def listas_para_dict(nomes, idades)` que recebe como parâmetros uma lista de strings (*nomes*) e uma lista de inteiros (*idades*). A função deve retornar um dicionário cujas chaves sejam os elementos da lista *nomes* e os valores devem ser elementos da lista *idades*, de maneira que *nomes*[0] seja a chave para *idades*[0], *nomes*[1] seja a chave para *idades*[1] e assim sucessivamente.

Exemplo:

```
1 nomes = ["joão", "maria", "josé", "mateus"]
2 idades = [12, 22, 45, 67]
3 dicionario = {
4     "joão" : 12,
5     "maria" : 22,
6     "josé" : 45,
7     "mateus": 67
8 }
```

12. (1 point) Escreva a função `def recebe_aluno()` que recebe via `input()` 3 valores: *nome* sendo uma *string*, *curso* sendo uma *string*, e *periodo* sendo um inteiro. A função deve retornar um dicionário contendo as chaves *nome*, *curso* e *periodo* e os valores sendo aqueles recebidos via `input()`. **Exemplo:**

```

1  dicionario = {
2      "nome": "arthur",
3      "curso": "ciência da computação",
4      "periodo": 8
5  }

```

13. (1 point) Escreva a função `def recebe_alunos(n)` que deve receber como parâmetro um inteiro n e deve retornar uma lista contendo n dicionários contendo as chaves *nome*, *curso* e *periodo* e os valores sendo aqueles recebidos via `input()`.

Obs.: Utilize a função `def recebe_aluno()` para receber um dicionário que representa um aluno.

Exemplo:

```

1  alunos = [
2      {
3          "nome": "arthur",
4          "curso": "ciência da computação",
5          "periodo": 8
6      },
7      {
8          "nome": "mauro",
9          "curso": "biologia",
10         "periodo": 2
11     }
12 ]

```

14. (1 point) Escreva um programa que analisa uma lista de dicionários que representam alunos com as chaves *nome*, *curso* e *periodo* e *printa* na tela qual o maior período dentre os alunos.

Exemplo:

```

1  alunos = [
2      {
3          "nome": "arthur",
4          "curso": "ciência da computação",
5          "periodo": 8
6      },
7      {
8          "nome": "mauro",
9          "curso": "biologia",
10         "periodo": 2
11     }
12 ]

```

Saída: 8